CS 550 Operating Systems
Spring 2026

Thread synchronization: locks

Overview

- The basic concept
- Metrics to evaluate lock implementation
- Lock implementations
- Using lock with `pthread`
- Mutex deadlocks

Lock: serializing concurrent critical section access

```
balance = balance + 1;
```

```
lock_t mutex;
...
lock(&mutex);
balance = balance + 1;
unlock(&mutex);
```

- A lock is a variable of some type.
- A lock (variable) can be either
  - available (or unlocked, or free): no thread holds the lock.
  - acquired (or locked, or held): exactly one thread is holding the lock.
- **lock()**: the calling thread is trying to acquire the lock. Succeeds if the lock is available, and the thread becomes the owner of the lock. Blocks or fails if the lock is acquired.
- **unlock()**: called by the owner of the lock to free an acquired lock.
- Basic task of lock: providing mutual exclusion (mutex).

Evaluating locks

- Functionality correctness – whether the lock does its basic task … Mutual exclusion
- Fairness – whether each thread contending for the lock gets a fair shot at acquiring it once it is free.
- Performance – time overheads added by using the lock.
  - When there is no contention
  - When multiple threads contending the lock on the same CPU
  - When multiple CPUs are involved

Root causes of race conditions

- Different threads within the same process share the same data section and heap.
- Shared data/heap can be read/written by threads within the same process *concurrently*.
  - How?
    - On multi-core processor, threads can be scheduled to run in different cores, so that threads are running, and thus accessing critical sections concurrently.
    - On single-core processor, preemption (i.e., non-cooperative scheduling) can cause different threads to execute the same critical section at the same time.

Mutual exclusion - disabling interrupts

- One of the earliest ways to provide mutual exclusion.
- Why can disabling interrupts can provide mutual exclusion?
- Pros: simplicity
- Cons?
  - It is a privileged operation that can be abused.
  - Does not work on multiprocessors.
  - Turning off interrupts for an extended period of time can lead to interrupt lost.
  - Inefficient (code that masks or unmasks interrupts tends to be executed slowly by modern CPUs).

```
void lock() {
    DisableInterrupts();
}
void unlock() {
    EnableInterrupts();
}
```

Mutual exclusion – setting/testing a flag

```
1  typedef struct __lock_t { int flag; } lock_t;

2  void init(lock_t *mutex) {
3      // 0 -> lock is available, 1 -> held
4      mutex->flag = 0;
5  }

6  void lock(lock_t *mutex) {
7      while (mutex->flag == 1)  // TEST the flag
8          ; // spin-wait (do nothing)
9      mutex->flag = 1;          // now SET it!
10 }

11 void unlock(lock_t *mutex) {
12     mutex->flag = 0;
13 }
```

Any problems?

Mutual exclusion – setting/testing a flag

Thread 1	Thread 2
call `lock()`	
while (flag == 1)	
interrupt: switch to Thread 2	
	call `lock()`
	while (flag == 1)
	flag = 1;
	interrupt: switch to Thread 1
flag = 1; // set flag to 1 (too!)	

- No mutual exclusion (violating the first metric for evaluating a lock).
- Bad performance (especially on a uniprocessor)
- How can we solve this?
  - We need help from
    - hardware to solve the mutual exclusion problem.
    - OS to solve the performance problem

Hardware support for implementing a spin lock

- The test and the set operations should be done in a atomic way to ensure correctness.
- Hardware instructions for this purpose.
  - Test-and-set
  - Compare-and-swap
  - Load-linked and store-conditional
  - Fetch-and-add

Test-and-set (a.k.a, atomic exchange)

- An instruction of achieving atomic testing and setting a variable.
- Different names in different architectures, e.g.,
  - `ldstub` on SPARC
  - `xchg` on x86
- The instruction basically does the following in an atomic way:

```
int TestAndSet(int *ptr, int new) {
    int old = *ptr; // fetch old value at ptr
    *ptr = new;     // store 'new' into ptr
    return old;     // return the old value
}
```

Spin lock implementation using test-and-set instruction

```
typedef struct __lock_t {
    int flag;
} lock_t;

void init(lock_t *lock) {
    // 0 indicates that lock is available, 1 that it is held
    lock->flag = 0;
}

                                          1       1
void lock(lock_t *lock) {
    while (TestAndSet(&lock->flag, ? ) == ? )
        ; // spin-wait (do nothing)
}

void unlock(lock_t *lock) {
    lock->flag = 0;
}
```

Compare-and-swap

- In some architectures, this instruction is called compare-and-exchange (e.g., x86 `CMPXCHG` instruction)
- The instruction does the following atomically:

```
int CompareAndSwap(int *ptr, int expected, int new) {
    int actual = *ptr;
    if (actual == expected)
        *ptr = new;
    return actual;
}
```

Spin lock implementation using compare-and-swap instruction

```
typedef struct __lock_t {
    int flag;
} lock_t;

void init(lock_t *lock) {
    // 0 indicates that lock is available, 1 that it is held
    lock->flag = 0;
}

void lock(lock_t *lock) {
    while (TestAndSet(&lock->flag, 1) == 1)
        ; // spin-wait (do nothing)
}

void unlock(lock_t *lock) {
    lock->flag = 0;
}
```

```
while (CompareAndSwap(&lock->flag, ? , ? ) == ? )
```

0 1 1

Why do we need “compare and swap” given that “test and set” can achieve the goal?

Load-linked and store conditional

```
int LoadLinked(int *ptr) {
    return *ptr;
}
```

Just like a typical load instruction, which fetches a value from memory and places it in a register

```
int StoreConditional(int *ptr, int value) {
    if (no one has updated *ptr since the LoadLinked to this address) {
        *ptr = value;
        return 1; // success!
    } else {
        return 0; // failed to update
    }
}
```

How can we use this LL/SC primitives to implement a spin lock?

Spin lock implementation using LL/SC

```
void lock(lock_t *lock) {
    while (1) {                                  1
        while (LoadLinked(&lock->flag) == ? )
            ; // spin until it's zero          1      1
        if (StoreConditional(&lock->flag, ? ) == ? )
            return; // if set-it-to-1 was a success: all done
                    // otherwise: try it all over again
    }
}

void unlock(lock_t *lock) {
    lock->flag = 0;
}
```

A simpler implementation:

```
void lock(lock_t *lock) {
  while (LoadLinked(&lock->flag)||!StoreConditional(&lock->flag, 1))
    ; // spin
}
```

Locks in pthread library: mutex

- A pthread mutex is basically a lock that we set (lock) before accessing a shared resource and release (unlock) when we're done. While it is set, any other thread that tries to set it will block until we release it.
- If more than one thread is blocked when we unlock the mutex, then all threads blocked on the lock will be made runnable, and the first one to run will be able to set the lock.
  - The others will see that the mutex is still locked and go back to waiting for it to become available again.

Pthread mutex

- In pthreads, a mutex variable is represented by the `pthread_mutex_t` data type.
- Before we can use a mutex variable, we must first initialize it by
  - either setting it to the constant `PTHREAD_MUTEX_INITIALIZER` (for statically allocated mutexes only) — e.g.,

```
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
```

  - or calling `pthread_mutex_init()`

Pthread mutex

```
#include <pthread.h>

int pthread_mutex_init(pthread_mutex_t *restrict mutex,
                       const pthread_mutexattr_t *restrict attr);

int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

Both return: 0 if OK, error number on failure

- To initialize a mutex with the default attributes, we set `attr` to `NULL`.

- If we allocate the mutex dynamically (by calling `malloc()`, for example), then we need to call `pthread_mutex_destroy()` before freeing the memory.

Pthread mutex

```
#include <pthread.h>

int pthread_mutex_lock(pthread_mutex_t *mutex);
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

Both return 0 on success, or a positive error number on error

- `pthread_mutex_lock()` - to lock a mutex
- `pthread_mutex_unlock()` - to unlock a mutex previously locked by the calling thread
- `pthread_mutex_trylock()` and `pthread_mutex_timedlock()`
  - If the mutex is currently locked, `pthread_mutex_trylock()` fails, returning the error EBUSY.
  - The caller can specify an additional argument that places a limit on the time that the thread will sleep while waiting to acquire the mutex.

Mutex deadlocks

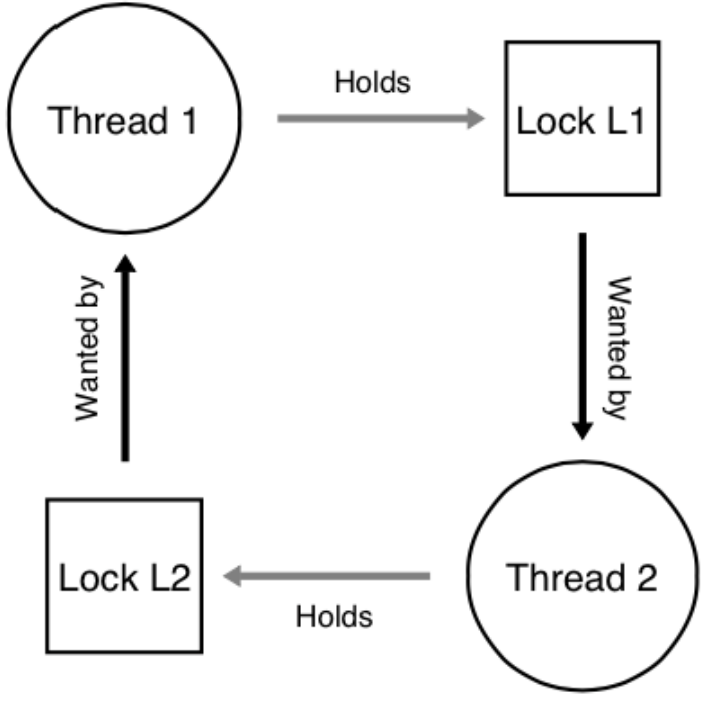
Thread A	**Thread B**
1. *pthread_mutex_lock(mutex1);*	1. *pthread_mutex_lock(mutex2);*
2. *pthread_mutex_lock(mutex2);*	2. *pthread_mutex_lock(mutex1);*
blocks	blocks

How to avoid mutex deadlocks?

Mutex deadlocks

```
Thread 1:
lock(L1);
lock(L2);
```

```
Thread 2:
lock(L2);
lock(L1);
```



Conditions for deadlock

- Four conditions need to hold for a deadlock to occur:
  - **Mutual exclusion**: Threads claim exclusive control of resources that they require (e.g., a thread grabs a lock).
  - **Hold-and-wait**: Threads hold resources allocated to them (e.g., locks that they have already acquired) while waiting for additional re- sources (e.g., locks that they wish to acquire).
  - **No preemption**: Resources (e.g., locks) cannot be forcibly removed from threads that are holding them.
  - **Circular wait**: There exists a circular chain of threads such that each thread holds one more resources (e.g., locks) that are being requested by the next thread in the chain.

Deadlock prevention

- Circular wait
  - Provide a total ordering on mutex acquisition (i.e., define a mutex hierachy): all the mutexes are locked and unlocked according to a certain order.
- Hold-and-wait
  - Use a deadlock prevention lock.
- No preemption
  - Try then back off approach.
    - I.e., `pthread_mutex_trylock()`
- Mutual exclusion
  - Lock-free data structure

```
lock(prevention);
lock(L1);
lock(L2);
...
unlock(prevention);
```